



LUNA LIVE CHAT · TRAVELGENIX

Luna Chat Staff Guide

The internal playbook for provisioning, launching and operating Luna for clients — from creating a client to running the knowledge pipeline and keeping it all green.

Internal — Travelgenix team

Provision · deploy · operate · troubleshoot

Audience: Travelgenix staff

Contents

1 Architecture overview

The moving parts and where each one lives

2 Onboarding a new client

The admin-gated Add New Client flow

3 Widget setup & deploy (staff side)

Connect and Deploy steps; the embed snippet

4 Environment variables

Every key, grouped by feature

5 File sharing setup

Connecting the Vercel Blob store

6 WhatsApp provisioning

Providers, webhook, number mapping

7 The knowledge pipeline & review

Global brain, crons, the review dashboard

8 Monitoring & health

The heartbeat and stats endpoints

9 Go-live checklist (staff)

Per-client launch sign-off

10 Troubleshooting

Common symptoms and their fixes

1 Architecture overview

Luna is an AI live-chat widget for travel businesses, branded **Travelgenix**. A visitor opens the chat bubble on a client's site, talks to Luna (the AI), and the conversation appears live in that client's agent dashboard. Luna answers from a shared travel knowledge base until it escalates or a human takes over. The same inbox also receives WhatsApp.

The pieces (and where each lives)

Piece	What it is
Vercel project <code>luna-chat-endpoint</code>	The whole product — serverless functions under <code>/api/*</code> and static pages under <code>/public/*</code> . Production: <code>luna-chat-endpoint.vercel.app</code> (also served on <code>chat.travelify.io</code>).
Airtable — Global brain	<code>appPKx77relfeiqmq</code> . Shared knowledge powering every client: <code>Knowledge</code> , <code>Destinations</code> , <code>Transport</code> (searched live, no status filter), plus staging/audit tables.
Airtable — Per-client CRM	<code>app60t3e0b3DangkB</code> . The client roster and conversations: <code>Clients</code> , per-client <code>Knowledge</code> , <code>Conversations</code> , <code>Messages</code> .
Ably (realtime)	Carries live messages between widget, server and dashboard. Each client uses their own Ably key.
Anthropic API	Generates Luna's replies (<code>api/luna-chat.js</code>) and the copilot.
Vercel Blob	Stores chat file/image attachments at stable public URLs (<code>api/upload.js</code>).
WhatsApp providers	Pluggable under <code>lib/providers/</code> (Gupshup default, 360dialog alternative).
Supabase pgvector	Optional semantic search (project <i>Luna Agents</i>). Off by default.



Tip. The single admin review hub for the shared knowledge base is `.../global-brain.html`, gated by the admin password.



Warning. The global brain is read live with *no status filter*. Anything written into `Knowledge` / `Destinations` / `Transport` is immediately visible to every client's Luna. Treat direct edits as going straight to production.



Two hosts, one project. `luna-chat-endpoint.vercel.app` (onboard/setup/dashboard URLs, AI endpoint) and `chat.travelify.io` (WhatsApp webhook, monitor alerts) both point at the same Vercel project.

2 Onboarding a new client

Onboarding happens at `/onboard.html`, behind the **Admin Access** password screen (checked against `ADMIN_PASSWORD`). New clients are written to the CRM base.

Client Onboarding
Set up a new client in seconds

Existing Clients
Click View to see their dashboard URL and embed code.
Loading...

Add New Client
Enter the client details. Everything else is generated automatically.

CLIENT NAME
Sunshine Travel

CLIENT SLUG (AUTO-GENERATED)
sunshine-travel

CONTACT EMAIL
hello@sunshinetravel.com

DASHBOARD PASSWORD
Su9\$kl2mQp

ABLY API KEY
aB12Cd.xY34Z:9fGhJkLmNoPqRsTuVwXyZ012

Each client needs their own Ably account for private conversations. Send them the Ably Setup Guide.

DEEP LINK SITE ID
250

The Travelgenix site ID for holiday search deep links. This enables the booking search feature in the chat widget.

Create Client

ONBOARD The **Add New Client** form. Everything except the fields shown is generated automatically.

Steps

- 1 Open `/onboard.html`, enter the **admin password**, then **Continue**.
- 2 Review the **Existing Clients** list — each shows name, slug and a status chip. **View** reopens that client's dashboard URL and embed code to re-share.
- 3 Complete every field in **Add New Client** (see table), then **Create Client**.
- 4 On success, copy the three boxes: **Dashboard URL**, **Widget Embed Code** and **Login Credentials**, and share them securely.

Field	What it is / where to get it
Client Name	The client's company name (e.g. <i>Sunshine Travel</i>). The canonical identity — embedded in the dashboard URL and the widget's <code>data-clientName</code> ; login and stats match on it. Saved as <code>ClientName</code> .
Client Slug	Read-only, auto-generated (lowercase, hyphenated). Used for namespacing (e.g. Blob paths). Saved as <code>ClientSlug</code> .
Contact Email	The client's contact address. Saved as <code>ContactEmail</code> .
Dashboard Password	Auto-generated; what the client types to log in. Saved as <code>DashboardPassword</code> .
Ably API Key	<i>The client's own</i> key (<code>xxxxx.yyyyy:zzzzzzzzzz</code>). They create an account at ably.com , make a Pub/Sub app and copy the API key. Each client needs their own so conversations stay private. Required.
Deep Link Site ID	The Travelgenix site ID for holiday-search deep links (e.g. <code>250</code>). Enables the in-chat booking search. Saved as <code>DeepLinkSiteID</code> . Optional.



Tip. If the client doesn't yet have an Ably key, send them the Ably Setup Guide first — you can't create the client without it.



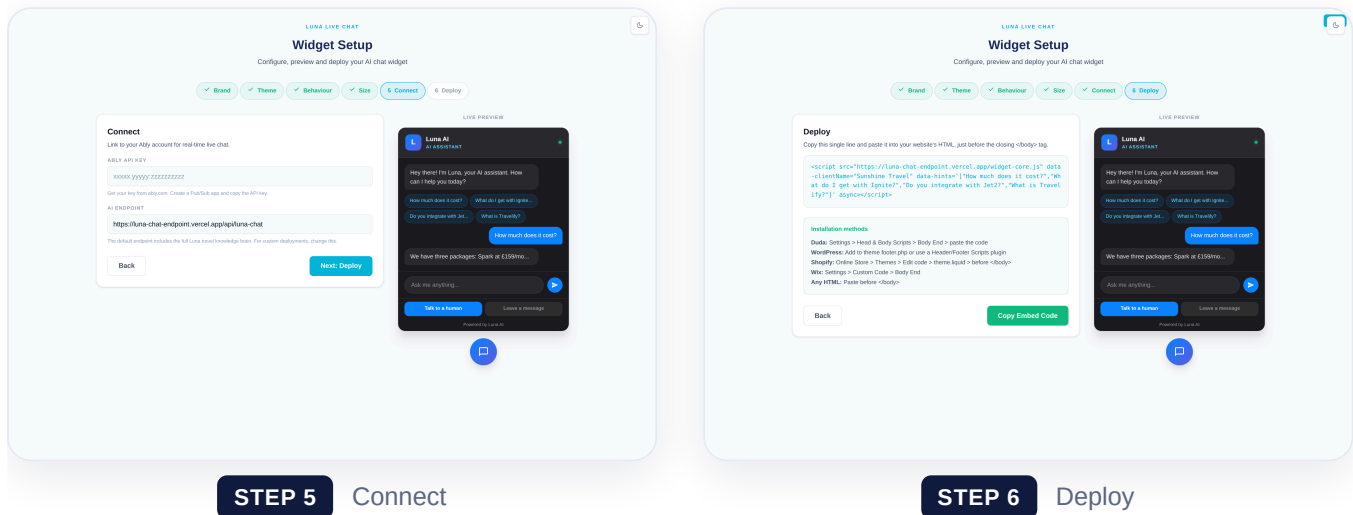
Warning. The generated Dashboard URL can contain the Ably key and password in the query string. Share it over a secure channel, never in a public ticket.



Warning. `ADMIN_PASSWORD` has an in-code fallback if unset — make sure it's actually set in Vercel.

3 Widget setup & deploy (staff side)

The visual configurator at `/setup.html` is a six-step wizard (Brand · Theme · Behaviour · Size · Connect · Deploy) with a live preview. Steps 1–4 are cosmetic; staff care most about **Connect** and **Deploy**.



Step 5 — Connect

- 1 **Ably API Key** — paste the client's key (`xxxxx.yyyyy:zzzzzzzzzz`). Becomes `data-ablyKey` on the embed and powers live handover. Without it the widget runs AI-only with no live inbox.
- 2 **AI Endpoint** — defaults to `.../api/luna-chat` , which includes the full Luna travel knowledge brain. Only change it for a custom deployment (maps to `data-endpoint`).

Step 6 — Deploy

- 1 The wizard renders a single-line **embed snippet** that only includes `data-*` attributes differing from defaults.
- 2 Click `Copy` .
- 3 Give the client the snippet to paste just before `</body>` (install methods for Duda, WordPress, Shopify, Wix and plain HTML are listed in the wizard).



Tip. The onboarding success panel's embed code is the minimal version (just `data-clientName` + `data-ablyKey`). Use `setup.html` when the client wants custom branding, colours, hints or window size.



Warning. `data-clientName` must *exactly* match the Client Name in Airtable (watch for trailing spaces / casing), or login, stats and conversation attribution won't resolve.



Warning. "Deploy" here means giving the client a snippet for their own site. It does *not* redeploy Vercel — that happens automatically on merge to `main` .

4 Environment variables

All env vars live on the `luna-chat-endpoint` Vercel project. Grouped by feature.

Core chat / realtime / admin

Var	Req?	What it does
<code>ANTHROPIC_API_KEY</code>	Yes	Anthropic key for Luna's replies and copilot.
<code>AIRTABLE_KEY</code>	Yes	Airtable PAT used by almost every endpoint.
<code>AIRTABLE_PAT</code>	Copilot	Alias used by the copilot endpoint.
<code>ABLY_ROOT_KEY</code>	Yes	Server-side Ably root key for publishing / token signing.
<code>ADMIN_PASSWORD</code>	Yes	Gates <code>/onboard.html</code> and <code>/global-brain.html</code> . Set it explicitly.
<code>CRON_SECRET</code>	Yes	Protects every cron endpoint and manual runs.
<code>LUNA_MODEL</code> / <code>LUNA_HAIKU_MODEL</code> / <code>LUNA_SHORT_MODEL</code>	Opt	Model overrides (in-code defaults used if unset).
<code>UPSTASH_REDIS_REST_URL</code> / <code>_TOKEN</code>	Rec.	Distributed rate limiting, monitor strike-state, WhatsApp dedupe + memory.

Copilot

Var	Req?	What it does
<code>ANTHROPIC_API_KEY</code> · <code>AIRTABLE_PAT</code>	Yes	Reused; reads the client's Luna Brain for grounded suggestions.
<code>LUNA_COPILOT_MODEL</code>	Opt	Defaults to a Haiku-class model.

Knowledge pipeline

Var	Req?	What it does
<code>TICKETMASTER_API_KEY</code>	Disc.	Events connector (auto-publishes).
<code>FOURSQUARE_API_KEY</code> / <code>FOURSQUARE_HOST</code>	Disc.	Things-to-do connector (host defaults to <code>places-api.foursquare.com</code>).
<code>TAVILY_API_KEY</code>	Reverify	Web search for grounded checks.
<code>LUNA_TRUSTED_DOMAINS</code>	Opt	Authoritative-source allowlist.
Tuning (optional)		<code>REVERIFY_LIMIT</code> , <code>CORRO_CAP</code> , <code>CONCURRENCY</code> , <code>AUTOAPPLY</code> , <code>DISCOVER_BATCH</code> , <code>FILL_*</code> , etc.

Email / SendGrid

Var	Req?	What it does
<code>SENDGRID_API_KEY</code>	Yes*	Daily review digest + transcript emails. Sender <code>noreply@travelgenix.io</code> is domain-authenticated. (*for digests/transcripts)
<code>REVIEW_DIGEST_TO</code> / <code>_FROM</code>	Opt	Recipient (default <code>andy.speight@agendas.group</code>) and sender.
<code>REVIEW_DASHBOARD_URL</code>	Opt	Link in the digest (defaults to the global-brain page).
<code>DIGEST_SKIP_WHEN_EMPTY</code>	Opt	<code>=1</code> suppresses the daily all-clear.

WhatsApp

Var	Req?	What it does
<code>WHATSAPP_PROVIDER</code>	No	<code>gupshup</code> (default) or <code>360dialog</code> .
<code>WHATSAPP_VERIFY_TOKEN</code>	Yes	Shared secret on the webhook URL (<code>?token=...</code> , fail-closed).
<code>WHATSAPP_NUMBER_MAP</code>	Yes	JSON mapping number/app → client record id.
<code>WHATSAPP_AI_AUTOREPLY</code>	No	<code>false</code> routes every WhatsApp chat to a human.
<code>GUPSHUP_API_KEY</code>	Gupshup	Account-level Gupshup key (or per-entry <code>apiKey</code>).
<code>WHATSAPP_D360_KEYS</code> / <code>_API_KEY</code>	360dialog	Per-number or global 360dialog key.

Semantic search (off by default) · File sharing · Monitoring

Var	Req?	What it does
<code>SEMANTIC_SEARCH</code>	Flag	<code>=1</code> turns hybrid retrieval on (needs keys below).
<code>VOYAGE_API_KEY</code> · <code>SUPABASE_URL</code> · <code>SUPABASE_SERVICE_KEY</code>	If on	Voyage embeddings + Supabase pgvector store.
<code>BLOB_READ_WRITE_TOKEN</code>	Uploads	Set automatically when a Vercel Blob store is linked. Without it, uploads return <code>503</code> .
<code>TELEGRAM_BOT_TOKEN</code> / <code>_CHAT_ID</code>	Alerts	Where the heartbeat sends down/recovery alerts.



Tip. `@sendgrid/mail`, `@vercel/blob` and `@anthropic-ai/sdk` are declared in `package.json` — no manual install on Vercel.

5 File sharing setup

Chat file/image sharing is stored in **Vercel Blob** (handler `api/upload.js`). It needs one one-time setup.

One-time setup (required)

- 1 In the **Vercel dashboard**, open the `luna-chat-endpoint` project.
- 2 Go to **Storage** → **Create / Connect** → **Blob**.
- 3 Connecting the store **automatically sets** `BLOB_READ_WRITE_TOKEN` — the only var the upload endpoint needs.

Limits

- **Max 4 MB** per file. The widget lets visitors pick images up to 15 MB and downscales them to ≤1600px client-side.
- **Allowed:** JPEG, PNG, GIF, WebP, PDF, DOC/DOCX, XLS/XLSX, CSV, TXT (anything else → `415`).
- Per-IP rate limited; unknown client → `404`; files namespaced `luna-chat/<slug>/<file>`.



Warning. If `BLOB_READ_WRITE_TOKEN` is not set, every upload returns **503 "File uploads are not configured"** and the attach button shows a friendly message — nothing else breaks. 503s on attach = Blob store not connected.



Warning. The Blob store is public-read by design. URLs are unguessable but not access-controlled — don't use chat file sharing for anything that must stay private.

6 WhatsApp provisioning

WhatsApp brings a client's own number into the **same inbox** as the web widget — multi-tenant, each client attributed to their own number. Inbound: `api/whatsapp-webhook.js`; outbound: `api/whatsapp-send.js`; provider logic under `lib/providers/`, selected by `WHATSAPP_PROVIDER`.

Provider choice

Provider	When	Cost shape
<code>gupshup</code> (default)	Demo / initial setup	No fixed fee; pay-as-you-go (~\$0.001/msg + Meta).
<code>360dialog</code>	Scale / clean exit	~£500–1,000/mo partner fee + ~€25/number. Easiest migration to direct Meta.

Start on **Gupshup** (cheapest to stand up). Switching later is a change to `WHATSAPP_PROVIDER` plus the number map — the orchestration code is unchanged.

Webhook URL

The provider POSTs inbound messages to:

```
https://chat.travelify.io/api/whatsapp-webhook?token=<WHATSAPP_VERIFY_TOKEN>
```

`GET` on that URL is the verification handshake. The token is verified fail-closed.

Per-client number mapping

`WHATSAPP_NUMBER_MAP` is JSON mapping each number/app to a client record id (`recXXXX`).

- **Gupshup** — keyed by app name: `{"LunaDemoApp": {"clientId":"rec...", "phone":"447700900000"}}`, plus `GUPSHUP_API_KEY`.
- **360dialog** — keyed by `phone_number_id`: `{"111222333": {"clientId":"rec...", "apiKey":"D360-..."}}`.

Conversation id is deterministic: `wa_<businessNumber>_<customerNumber>`. The same Ably channels carry it, so the dashboard shows it with a **WhatsApp** badge and no special handling.

Going live (Gupshup demo)

- 1 Create a Gupshup account → a WhatsApp app (sandbox to test instantly, or a real number).

2

Copy the API key → `GUPSHUP_API_KEY`. Note the app name and source number.

3

Set `WHATSAPP_PROVIDER=gupshup`, `WHATSAPP_VERIFY_TOKEN`, `WHATSAPP_NUMBER_MAP`.

4

In Gupshup, set the inbound/callback URL to the webhook URL above.

5

Message the number — the chat should appear in the inbox; Luna replies on the phone; an agent reply reaches the phone.



Tip. Set up Upstash. Without it, provider retries aren't de-duplicated and Luna loses multi-turn memory (each message answered statelessly).



Warning — 24-hour window. Free-form replies are only allowed within 24h of the customer's last message. Outside it you need approved templates, which are *not* wired up here.



Warning. Use real `rec...` ids and keep `WHATSAPP_NUMBER_MAP` valid JSON. A bad map means inbound messages can't be attributed and won't appear.

7 The knowledge pipeline & review

The pipeline keeps the **global brain** fresh and accurate, with a strong human-review gate for anything a model produced.

Prime directives (never break)

- **Zero hallucination.** Facts come only from trusted/fetched sources, never the model's memory. Anything that can't be grounded is flagged, never invented. Fail safe — an ambiguous reply becomes "unverifiable", never a false confirm.
- **Voice:** UK English, no em-dashes, no Oxford commas, plain and warm.
- **FCDO guardrail:** Luna never voices a safety assessment; the server attaches the official FCDO status as a card.
- **Auto-publish policy:** deterministic connector data (Ticketmaster/Foursquare, *no LLM*) auto-publishes to live Knowledge; anything a model *extracted* stays human-gated.

The review dashboard — </global-brain.html>

Admin-gated. One page, three tabs (each with a count badge): **Discovery suggestions** (approve/dismiss scraped suggestions), **Re-verification queue** (apply/dismiss re-checked or gap-filled facts), and **+ Add data** (add a destination or an airport/airline/cruise; the Search Index builds on save).
Per-client review: </luna-brain.html?client=<name>>.

Global Knowledge Review

Review what the discovery and re-verification jobs found for the shared knowledge base. Check each is accurate, then apply. Nothing here is live until you act on it.

0

suggestions pending

0

re-verifications pending

—

live knowledge records

—

oldest last verified

—

past review

Discovery suggestions 0

Re-verification queue 0

[+ Add data](#)

Add a destination, city or resort

Name is enough to start — the discovery crawler auto-fills things to do and events on its next run (usually within 3 hours). Fill any facts you already know; leave the rest blank. The search index is built for you, so it's findable straight away.

NAME *

e.g. Crete

COUNTRY

e.g. Greece (defaults to Name)

TYPE

Country

REGION

e.g. Europe

PARENT

City -- Country, Resort -- City

ISO CODE

e.g. GR

CURRENCY

CAPITAL

TIME ZONE

DIALLING CODE

EMERGENCY NUMBER

PLUG TYPE

VOLTAGE

UK VISA REQUIRED

TAP WATER SAFE

DRIVING SIDE

MAIN AIRPORT

Add destination

Add an airport, airline or cruise

These aren't auto-crawled, so the facts you enter here are exactly what Luna will serve. Use the Info boxes for the details — for an airport: terminals, transfers, lounges, accessibility, address.

NAME *

e.g. Faro Airport

TYPE *

Airport

CODE (IATA)

e.g. FAO

COUNTRY

CITY

KEY TIPS

INFO 1

INFO 2

INFO 3

INFO 4

INFO 5

INFO 6

Add transport

GLOBAL BRAIN

The **+ Add data** tab. Add a destination (name is enough — discovery auto-fills the rest) or a transport record (what you type is what Luna serves).

Crons (from `vercel.json`)

Cron	Schedule	Purpose
<code>/api/monitor</code>	<code>* /15 * * *</code> <code>*</code>	Heartbeat (checks A/B/C).
<code>/api/reverify-structured</code>	<code>0 2 * * *</code>	Re-check structured Destination/Transport fields via corroboration.
<code>/api/fill-gaps</code>	<code>0 3 * * *</code>	Extract blank structured fields from sources; stage as Pending (never auto-applies).
<code>/api/refresh-fcdo</code>	<code>0 4 * * *</code>	Write official FCDO status verbatim, no LLM.
<code>/api/reverify</code>	<code>0 5 * * *</code>	Q&A re-verification; 1 authoritative or 2 corroborating sources → auto-apply, else Pending.
<code>/api/backfill-search-index</code>	<code>30 5 * *</code> <code>*</code>	Rebuild empty Search Index + gap report.
<code>/api/review-digest</code>	<code>0 6 * * *</code>	Daily digest email of both Pending queues.
<code>/api/knowledge-freshness</code>	<code>0 8 * * *</code>	Staleness digest (global + per-client).
<code>/api/embed-index</code>	<code>0 9 * * *</code>	Build/refresh semantic-search vectors.
<code>/api/discover-scan</code>	<code>0 */3 * *</code> <code>*</code>	~230 Destinations, batch-25 rotation: curated scrape → Pending; connectors → auto-publish.

Daily review digest

Sent by `/api/review-digest.js` at 06:00 to `REVIEW_DIGEST_TO` , linking the global-brain page.
Sends every day including an "all clear" unless `DIGEST_SKIP_WHEN_EMPTY=1` . `?dryRun=1` previews JSON; `?force=1` forces a send.

Turning on semantic search

- 1 Create a Voyage key → `VOYAGE_API_KEY` .
- 2 Copy the Supabase service_role key → `SUPABASE_SERVICE_KEY` ; set `SUPABASE_URL` .
- 3 Run `/api/embed-index?secret=<CRON_SECRET>` once (or wait for the 09:00 cron).
- 4 Set `SEMANTIC_SEARCH=1` . Unset it to flip off instantly.



Tip. Retrieval runs keyword + semantic in parallel and merges. Any error or missing config silently falls back to keyword search, so semantic search can't break a reply.



Warning. Approving items writes to *live* Knowledge. Only approve grounded, sourced facts — never bypass the human gate for model-extracted content.

8 Monitoring & health

Heartbeat — `api/monitor.js`

Runs every 15 minutes (cron-only; manual with `?secret=<CRON_SECRET>`). Three checks in parallel:

- **Check A — Visitor path:** POSTs `/api/ably-token` like a widget. Catches a stuck rate limiter or dead signing key.
- **Check B — Dashboard path:** authenticates Travelgenix's dashboard Ably key. Catches a revoked/dead key.
- **Check C — AI path:** a tiny real Anthropic call against each distinct model Luna uses. Catches a retired/renamed model id, a bad key or an outage.

Alerts fire only after **two consecutive failures** (~15 min real outage): one "down" Telegram, silence, then one "recovered". Strike-state lives in Upstash. Returns 200 when healthy, 503 when not.



Tip. Test alert wiring without an outage: `GET /api/monitor?secret=<CRON_SECRET>&test=alert`.

Stats — `api/luna-stats.js`

Powers the dashboard Stats panel. `GET` with `X-Client-Name` (or `?client=`). Computes totals, AI-only vs escalated split, avg duration/messages, visitor CSAT with distribution, last-14-days counts, peak hour, top pages and top topics.

Go-live checklist (staff)

Run through this per client before declaring them live.

- ☐ **Client created** in `/onboard.html` (appears in Existing Clients; all fields populated).
- ☐ **Ably connected** — client's own key saved and present in the embed (`data-ablyKey`).
- ☐ **Widget deployed** on the client's site (snippet before `</body>` ; `data-clientName` matches Airtable exactly; bubble loads, Luna responds).
- ☐ **Login works** — client signs in; a live test chat appears; agent reply reaches the widget.
- ☐ **Knowledge seeded** — relevant destinations/transport present; review digest landing.
- ☐ **WhatsApp live (if used)** — number map entry set, callback URL with `?token=` , end-to-end test passes.
- ☐ **Blob connected (if file sharing)** — `BLOB_READ_WRITE_TOKEN` set; attach uploads without 503.
- ☐ **Digests landing** at `REVIEW_DIGEST_TO` .
- ☐ **Monitor green** — `/api/monitor` returns `healthy: true` ; Telegram alerts confirmed.

10 Troubleshooting

Symptom	Likely cause	Fix
Attach shows "uploads not configured" / upload 503	Blob store not linked	Link a Blob store (Section 5).
Upload 413	File over 4 MB	Re-send under 4 MB.
Upload 415	Unsupported type	Use an allowed type.
Upload 404 "Unknown client"	clientName mismatch	Check the widget's data-clientName matches ClientName.
Every chat reply fails / chat 404	Anthropic model id retired/renamed	Check LUNA_*_MODEL ; monitor Check C reports the model + 404.
Copilot 503	Missing ANTHROPIC_API_KEY / AIRTABLE_PAT	Set copilot env vars; confirm allowed origin.
429 on chat/token/upload	Rate limit / stuck limiter	Wait; verify Upstash healthy (monitor Check A flags a stuck limiter).
Dashboard can't connect / no live chat	Dashboard Ably key revoked	Monitor Check B reports it; re-issue the key in Airtable.
Onboard/global-brain 401	Wrong admin password / unset	Set ADMIN_PASSWORD ; use the correct value.
Client login fails	Name mismatch or wrong password	Confirm ClientName casing/spaces and saved DashboardPassword.
WhatsApp never reaches inbox	Bad number map / wrong token / dedupe failing	Validate JSON map + rec ids; confirm ?token= ; ensure Upstash set.
WhatsApp reply not sent	Outside 24h window or missing key	Reply within 24h; set the provider API key.
New Airtable rows not found	Empty Search Index	05:30 backfill indexes them; or run it manually (?all=1).
Semantic search "silently off"	Flag unset / keys missing / not populated	By design (falls back to keyword). Follow Section 7 to enable.
No daily digest	SENDGRID_API_KEY unset or skip-when-empty	Set the key; ?force=1 to force, ?dryRun=1 to preview.
Stats show zeros	No conversations / wrong client id	Confirm X-Client-Name matches ClientName.

Internal use · **Luna Chat** · Travelgenix

Keep the prime directives intact — when in doubt, fail safe.